Bellman-Smoothed Regression for Value Function Estimation

Anonymous Authors[1]

Abstract

Value function estimation is a core problem in Reinforcement Learning. Monte-Carlo methods compute an unbiased estimate of the value function, but can suffer variance related to the length and noisiness of the task. Temporal differences methods (TD Learning) are popular in part because they reduce variance, and in part because they induce local Bellman-consistency of the learned value function. However, TD learning can fail to converge when using non-linear function approximation. We study a new learning rule based on the minimization of a convex Bellman-smoothed value loss. We extend previous results that show when TD learning succeeds at lowering the Bellman-smoothed value loss. Our main contribution is using this loss for value function learning in deep RL. We first show how to estimate the new objective in an unbiased fashion from trajectory data – with strictly lower variance than Monte Carlo. We show that in expectation, the learned value function leads to stronger policy improvement than classical TD learning. We experiment on RL tasks, including continuous control, showing that our value function loss improves the performance of PPO.

1. Introduction

2. Preliminaries

$$D \doteq \text{diag}(\boldsymbol{\mu}) \tag{1}$$
$$A \doteq D(I - \gamma P) \tag{2}$$

A is positive definite (TV 1997). We make use of the labels S and K for its symmetric and skew symmetric components respectively.

$$A = \frac{1}{2}(A + A^\intercal) + \frac{1}{2}(A - A^\intercal) \tag{3}$$
$$= S + K \tag{4}$$

The objective is to train a differentiable value estimate v_θ, with $|\theta| = k$. Writing the estimate at each state gives the estimated value vector $\boldsymbol{v}$. The value error vector is given by

$$\boldsymbol{e} \doteq (\boldsymbol{V} - \boldsymbol{v}). \tag{5}$$

The fundamental goal of policy evaluation is to improve the policy. The instrumental goal is to minimize the following error, called the MSE.

$$\mathcal{L}_{MC} \doteq \|\boldsymbol{e}\|_D^2. \tag{6}$$

Monte Carlo value learning explicitly minimizes **??**, as estimated in an unbiased fashion from sampled rollouts. In contrast, bootstrapping methods like Temporal Difference (TD) Learning (**?**) instead use the notion of temporal difference error, given by: $\boldsymbol{\delta} = \boldsymbol{R} + \gamma P \boldsymbol{v} - \boldsymbol{v}$ TD learning is an iterative algorithm that is based on relationship between the TD error and the gradient of the value estimator. We will write its expected update in matrix form, stacking the gradients at each state into the $N \times k$ Jacobian matrix, J_θ:

$$J_\theta \doteq \begin{bmatrix} \nabla v_\theta(s_1)^\top \\ \nabla v_\theta(s_2)^\top \\ \vdots \\ \nabla v_\theta(s_N)^\top \end{bmatrix} \in \mathbb{R}^{N \times k}. \tag{7}$$

In expectation, the TD update and loss are given by the following two equations.

$$\boldsymbol{\theta} \xleftarrow{TD} \boldsymbol{\theta} + \alpha J_\theta^\intercal D \boldsymbol{\delta} \tag{8}$$
$$\mathcal{L}_{TD} \doteq \|\boldsymbol{R} + P\overline{\boldsymbol{v}} - \boldsymbol{v}\|_D^2 \tag{9}$$

Where $\overline{f_\theta}$ indicates the stop gradient operator is applied to θ. A *TD Fixed Point* is a value of θ where no update occurs in expectation. A TD fixed point is reached whenever $\mathbf{0}_k = J_\theta^\intercal D \boldsymbol{\delta} = \mathbb{E}_{s \sim \mu}(\delta(s) \nabla v(s))$.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

It is known (**?**) that (**??**) can be rewritten using the substitution $A\boldsymbol{e} = D(I-\gamma P)(\boldsymbol{V}-\boldsymbol{v}) = D(\boldsymbol{R}-(I-\gamma P)\boldsymbol{v} = D\boldsymbol{\delta}$. This substitution is a key insight from (**?**)'s analysis of deep TD learning, and means (**??**) can be rewritten as:

$$\boldsymbol{\theta} \xleftarrow{TD} \boldsymbol{\theta} + \alpha J_\theta^\intercal A\boldsymbol{e} \tag{10}$$

3. Related Work

and munos, Tsis- nd Van Roy, 2016,

4. Policy Optimization Bellman-Smoothed Value Error

The Bellman-Smoothed value error from (**?**) is given by:

$$E \doteq \|\boldsymbol{V} - \boldsymbol{v}\|_S^2 = \boldsymbol{e}^\intercal A\boldsymbol{e} = \boldsymbol{e}^\intercal S\boldsymbol{e}. \tag{11}$$

It's been shown ((**?**), (**?**)) that A is positive definite when μ is the stationary distribution for the on-policy Markov Chain. This means that $E \geq 0$, and it equals zero only when the value error is zero at all visited states. We study gradient descent on E, which makes the following update:

$$\boldsymbol{\theta} \xleftarrow{E} \boldsymbol{\theta} + \alpha J_\theta^\intercal S\boldsymbol{e} \tag{12}$$

Proposition 4.1. *The expected update due to gradient descent on E is equal to the expected update due to TD learning, less $J_\theta^\intercal K\boldsymbol{e}$, where $K = \frac{1}{2}(A - A^\intercal)$ is the skew-symmetric component of A.*

Proof.

$$J_\theta^\intercal S\boldsymbol{e} = J_\theta^\intercal (A - K)\boldsymbol{e} = J_\theta^\intercal A\boldsymbol{e} - J_\theta^\intercal K\boldsymbol{e} \tag{13}$$

□

Since the Bellman-Smoothed Value Error is the S-weighted norm of the value error vector, E can be treated as a squared error. It is convex in $\boldsymbol{v}$, and minimized when the value error is zero at all visited states. Under typical assumptions gradient descent can be shown to converge to a local minimum of E (as in supervised learning). In contrast, exact TD learning may not converge. Moreover, its fixed points may have positive value error (**?**). This is due the of the skew-symmetric component K in the TD update. A TD fixed point can have non-zero value error if $J_\theta^\intercal S\boldsymbol{e} = -J_\theta^\intercal K\boldsymbol{e}$. The presense of K also implies that the TD-Learning update (**??**) can increase the value error E, possibly leading to divergence [TODO: prove]. (**?**) showed that when the Markov Chain is reversible ($DP = (DP)^\intercal$), TD learning is equivalent to gradient descent on E.[1] However, in practical RL, the problem is rarely reversible.

Despite these theoretical short-comings, on-policy TD learning is popular. For one, the sampled update (e.g. the Monte Carlo estimator of (**??**)) has low variance due to only incorporating one-step noise. Secondly, with tabular and linear function approximators, the TD-fixed point is the same as the model-based solution are the same – this is the certainty equivalence principle (**?**), (**?**)). Third, TD-learning ascribes similar values to neighboring states, while Monte Carlo treats each state as disconnected.

As discussed later, E-minimization smooths the value function in a similar manner to TD learning, while remaining a safer optimization target. In the next section, we present empirical evidence that the exact version of E minimization compares favorably to exact TD and exact MC. Additionally, we present a form of E that can be sampled, with variance lying exactly between that of TD and MC.

Experiment: Comparing Expected Versions of the Algorithms We find empirically that TD learning performs worse than E minimzation when the Markov Chain has a significant skew-symmetric component. We study the behavior of E and $\mathcal{L}_{TD}$ and on nine environments, which are small enough to solve exactly.

- **Four Rooms (Sparse)**: 104 states (105 including terminal). A classic sparse reward navigation task on a 13×13 grid with 104 traversable cells partitioned into four rooms. The state representation is a grid, and a CNN is used for the value network.
- **Four Rooms + Mines (Sparse and Dense)**: Adds mines which terminate the agent's life. Dense versions of navigation tasks use potential-based reward shaping based on distance to the goal.
- **Eight Rooms (Sparse and Dense)**: 210 states (211 including terminal). An extended navigation task on a 25×13 grid with eight rooms.
- **Mountain Car (Sparse and Dense)**: A continuous state space (discreteized for tabular solvers). Dense rewards are obtained with potential (height)-based reward shaping.
- **Space Invaders**: 1,974 states (1,975 including terminal). A tabular 3-alien shielded-fleet version of Space Invaders on a 7×6 grid, with endless waves and termination only on death.
- **Whirlpool**: 105 states (105 including terminal). A gridworld using the same visual state representation as Four Rooms. The agent starts in the outermost part of the grid, and must navigate to the center of the whirlpool. A strong force pushes the agent clockwise, in the sense that the agent's action succeeds only 10% of the time, and 90% of the time the agent is pushed clockwise.

[1] In the reversible case A is symmetric and K is all zeros.

As we shall see, Whirlpool, Space Invaders, and Mountain Car all have strong skew-symmetry. Informally, these MDPs are far from reversible. A reversible MDP has the same chance of transitioning "forward" from $s \to s'$ as it does transitioning "backward" from $s' \to s$. In skew-symmetric MDPs, either the dynamics move only forward, or a strong asymmetric force acts on the agent (e.g. gravity in Mountain Car).

The above environments are small enough that we can exactly compute the TD update (**??**), the MSE loss (**??**), and Bellman-Smoothed Value Error (**??**). This removes any noise in the updates. We use deep value function approximation, using a CNN on visual tasks and a MLP on Mountain Car. The only randomness across runs is from neural network initialization. Hyperparamters can be found in the Appendix [TODO: put in hypers]. We first consider fixed policy evaluation. For each algorithm, we train a neural network to estimate the value of the uniform random policy. We denote the size of the final hidden layer by k, and try $k = 8$ and $k = 64$ to understand the impact of the capacity of the network. Since the Markov Chain is fixed, we can compute its skew-symmetry as the ratio of $\|K\|$ to $\|S\|$. For a reversible Markov Chain, $\|K\|$ will be zero. We plot this skew-symmetry measure against the relative error – the average MSE during learning for E relative to TD. A lower relative error means E performed better. Note that we use the on-policy MSE, which is the typical objective in policy evaluation, though neither algorithm minimizes it directly.

Figure **??** shows that, as Proposition **??** suggests, when K is small compared to S, E minimization and TD have the same MSE during learning. This is because their learning dynamics are almost the same, as Proposition **??** suggests. However, as the environment becomes more skew-symmetric, TD learning's performance declines relative to E minimization.

While the MSE is the de-facto objective for fixed policy evaluation, the goal of policy evaluation is policy improvement. We run the three algorithms to evaluate the uniform random policy. In addition to TD and E-minimzation, we also consider **Exact Monte Carlo** (MC), which is the minimzation of $\mathcal{L}_{MC}$ directly by fitting the true value function in a supervised fashion. Despite the name "Monte Carlo", the exact MC update has no noise besides random neural network initialization.

Results are shown in **??**. Exact MC usually obtains the best MSE by the end of learning, which makes sense as the MSE is its learning objective. We note that E-minimization learns the value function quickly – as measured by the training AUC of the MSE. We also consider two metrics designed to inform us of the quality of the learned value function for the purposes of policy improvement. Metric one is the start-state value of the greedy policy on the learned value function. This is the average return that would be obtained following the geredy policy. Metric one measures the strength of the policy due to a single round of approximate policy iteration. Metric two measures the quality of the policy gradient: the policy gradient can be shifted or scaled by a constant without affecting the policy update. As such, we measure the cosine-similarity, or normalized dot product, of estimated advantage and true advantage (the advantage is tread as a vector in state-action space). Since we learn value functions, we compute the estimated advantage as $\boldsymbol{a} = \boldsymbol{R} + \gamma P \boldsymbol{V}$. We observe that by both of these metrics, exact E-minimization learns a value function that is better for policy improvement.

Expected PPO. To see if these results carry over to policy optimization, we implement a no-sampling version of PPO, with a critic is due to exact E, exact TD, or exact MC. The policy is paramterized by an actor network with the same architecture as the critic, without parameter sharing between actor and critic. We use a constant learning rate for the actor, but tune the critic learning rate for each algorithm. Results for the best tuned setting are shown in Figure **??**. Unlike the fixed policy evaluation task, where a critic is trained to evaluate a fixed policy, PPO updates both the actor and critic at each round. This means that the learning algorithm must be able to evaluate a new policy quickly, with transfer between policies. In general, we see exact TD learning struggles with sparse rewards (see Eightrooms Sparse and Dense), possibly because it would require more iterations to propegate value through the state space. However, exact TD also struggles with dense, skew-symmetric tasks like Dense Mountain Car and Space Invaders.

References

Parr, R., Li, L., Taylor, G., Painter-Wakefield, C., and Littman, M. L. An analysis of linear models, linear value-function approximation, and feature selection for reinforcement learning. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pp. 752–759. ACM, 2008. doi: 10.1145/1390156.1390251.

Sutton, R. S. Learning to predict by the methods of temporal differences. *Mach. Learn.*, 3(1):9–44, August 1988. ISSN 0885-6125. doi: 10.1023/A:1022633531479. URL https://doi.org/10.1023/A:1022633531479.

Sutton, R. S., Mahmood, A. R., and White, M. An emphatic approach to the problem of off-policy temporal-difference learning. *Journal of Machine Learning Research*, 17(73): 1–29, 2016. URL https://jmlr.org/papers/v17/14-488.html.

Tang, Y. and Munos, R. Towards a better understanding of representation dynamics under td-learning. In *International Conference on Machine*

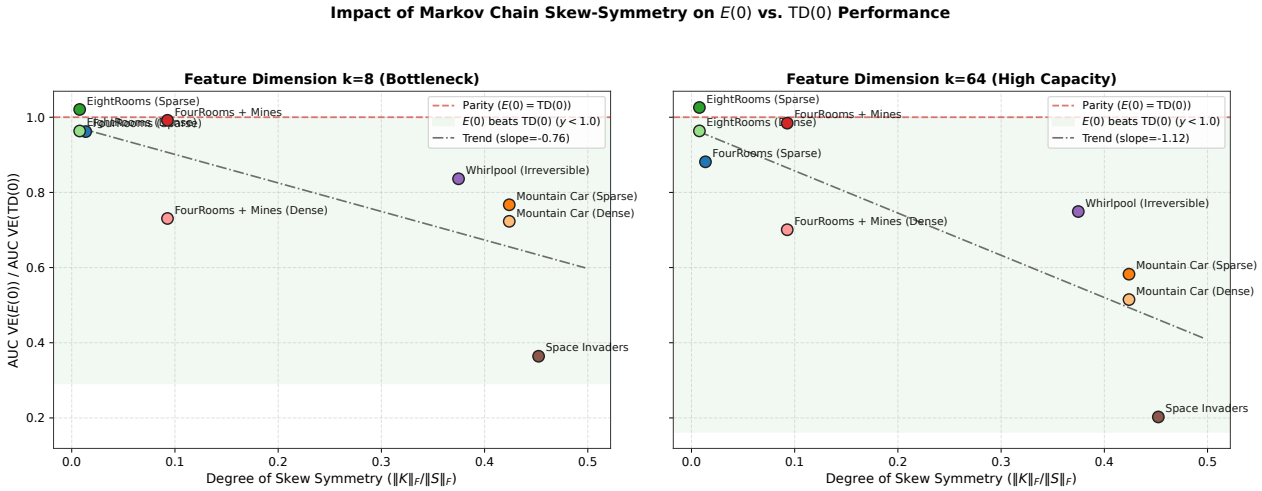


Figure 1. **Alignmnent vs. Skew Symmetry** Environments charted by their alignment based on the ratio of the norm of K to the norm of S and the relative performance of exact TD vs. exact E-Minimization $\frac{\text{AUC}(MSE_E)}{\text{AUC}(MSE_{TD})}$. The negative slope captures the fact that in more skew-symmetric environments, E has better value learning. 10 seeds.

Algorithm	MSE (AUC)	MSE (Final)	Advantage Acc.	Value of Greedy Policy
TD	(0.0/9) [0w+0t]	(0.3/9) [0w+1t]	(0.0/9) [0w+0t]	(2.0/9) [2w+0t]
MSE Minimization	**(4.5/9) [4w+1t]**	**(7.3/9) [7w+1t]**	(3.0/9) [2w+2t]	(3.0/9) [2w+2t]
E Minimization	**(4.5/9) [4w+1t]**	(1.3/9) [1w+1t]	**(6.0/9) [5w+2t]**	**(4.0/9) [3w+2t]**

Table 1. Win Rate of TD, MC, and E minimization across several metrics: The AUC and final MSE as well as two measures of policy improvement: (1) the cosine similarity between the learned and true advantage functions, and (2) the value of the greedy policy at the start staet. For each environment, an algorithm is a winner if it performs best on a certain metric. Ties occur when no single algorithm was statistically significantly best, and are split evenly. Thus $1.5/9$ indicates that the algorithm won outright on exactly one environment and tied on another.

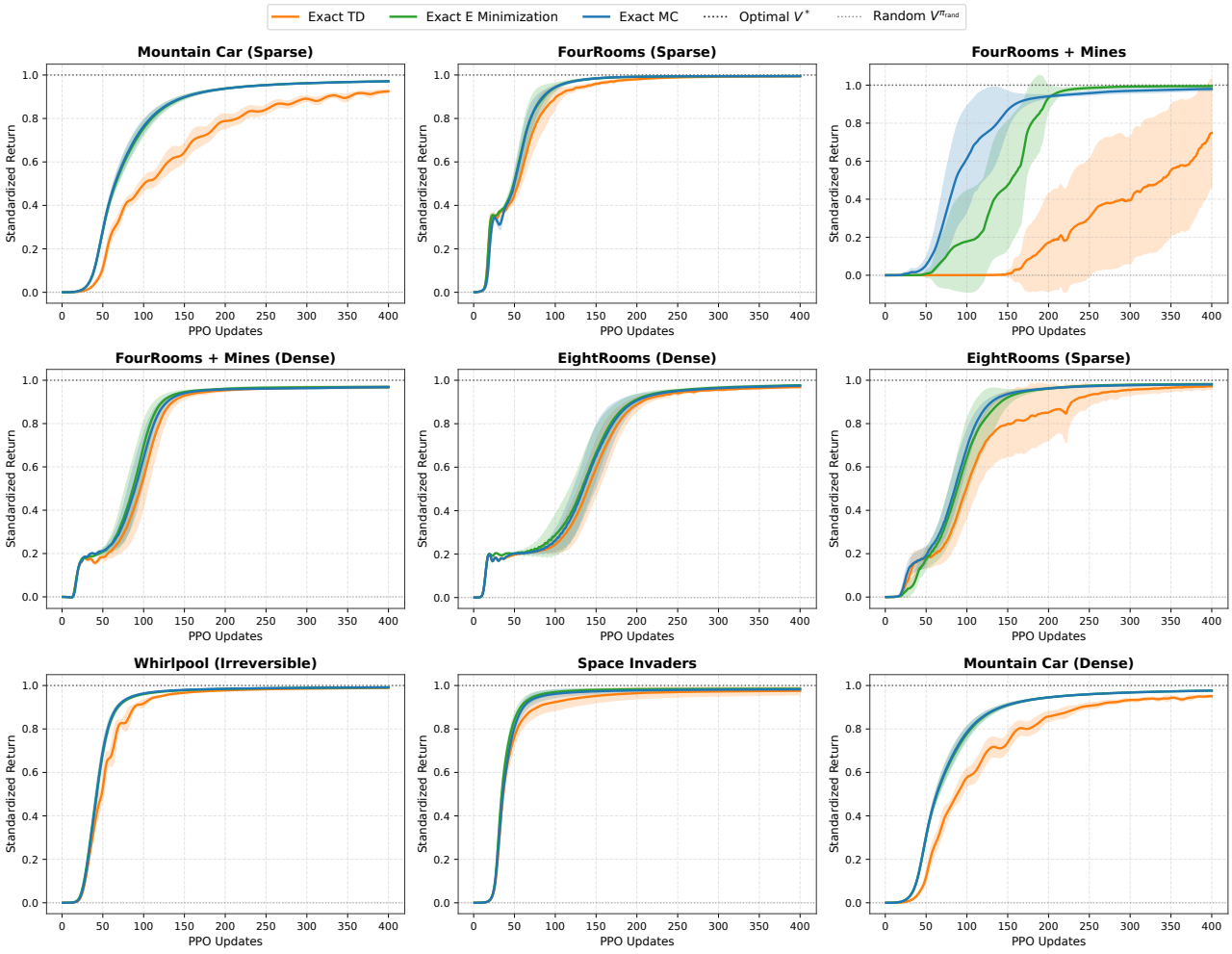


Figure 2. **Exact PPO.** Learning curves for no-sampling PPO with the critic due to exact TD, MC, or E-minimization. 10 seeds.

Learning (ICML), volume 202 of *Proceedings of Machine Learning Research*, pp. 33720–33738. PMLR, 2023. URL https://proceedings.mlr.press/v202/tang23g.html.

Tsitsiklis, J. N. and Roy, B. V. An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, 42(5):674–690, 1997. doi: 10.1109/9.580874.

A. You *can* have an appendix here.

B. Sampling the Bellman-Smoothed Value Error

Theorem B.1. *The Bellman-Smoothed Value Error can be written as*

$$E = (1-\gamma)\|\boldsymbol{e}\|_D^2 + \gamma\mathcal{E}, \tag{14}$$

where the energy, $\mathcal{E}$, is given by the following.

$$\begin{aligned}\mathcal{E} &= \frac{1}{2}\sum_{ij}\mu_i p_{ij}(e_i - e_j)^2 \\ &= \frac{1}{2}\sum_{ij}\mu_i p_{ij}(\underbrace{r_i + \gamma v(s_j) - v(s_i)}_{=\delta_i} + (1-\gamma)e_j)^2\end{aligned} \tag{15}$$

Equation **??** emphasizes that the energy $\mathcal{E}$ is the sum of squared terms, each of which can be estimated on-policy. Each squared term is the sum of the sampled TD error and *next state* Monte-Carlo error. Notice that the total weight on Monte-Carlo terms is $O((1-\gamma)^{-1})$, reducing their variance.

Proposition B.2. *The Bellman-Smoothed Value Error is equal to the D-weighted inner product of the value error and the TD-error.*

$$E = \boldsymbol{e}^\intercal D\boldsymbol{\delta} \tag{16}$$

Corollary B.3. *The magnitude of E is bounded by the magnitude of the Monte-Carlo and TD losses.*

$$E = \langle \boldsymbol{e}, \boldsymbol{\delta}\rangle_D \leq \|\boldsymbol{e}\|_D\|\boldsymbol{\delta}\|_D \tag{17}$$

One can obtain an unbiased sample of e_i using the discounted return $G_i \doteq \sum_{t\geq 0}\gamma^t r_t$.

add detail

Truncation Bias When sampling E using truncated rollouts of length T rather than episodic episodes, one can use the T-step bootstrapped return, leading to the following approximation of e_i

$$G_i^T = \sum_{t=0}^{T-1}\gamma^t r_t + \gamma^T\overline{v}(s_T)|s_0 = s_i e_i \approx \delta_i^T = G_i^T - \overline{v}(s_i) \tag{18}$$

The quality of the approximation in Equation **??** improves with T, becoming exact as $T\to\infty$.

We call the bias in estimating the the Bellman Smoothed Value Error with δ_i^T instead of e_i *truncation bias*. Using the Generalized Advantage Estimation (GAE) with $\lambda = 1$ on fixed-length rollouts will result in truncation bias increasing with the position in the rollout, finally resulting in e_T being approximated by δ_t. Examining the energy for this final transition:

$$\mathcal{E}_T \approx \frac{1}{2}(G_T - v(s_T) - G_{T+1} + v(s_{T+1}))^2 \tag{19}$$

$$\approx \frac{1}{2}(r_T + \gamma\overline{v}(s_{T+1}) - v(s_T) - (v(s_{T+1}) - v(s_{T+1})))^2 \tag{20}$$

$$\approx \frac{1}{2}(r_T + \gamma\overline{v}(s_{T+1}) - v(s_T))^2 \tag{21}$$

That is, the estimate of E obtained at the final step in the batch reverts to $\mathcal{L}_{TD}$.

Experiment: Truncation Bias We examine how truncation bias scales on small environments. We run PPO on each environment. PPO's update step first collects B parallel trajectories of length T, and then updates the actor and critic for a number of gradient steps. At each update step, we compute the expectation of the gradient $\boldsymbol{g}^* = \mathbb{E}_\mu\nabla_\theta E$, and the estimate

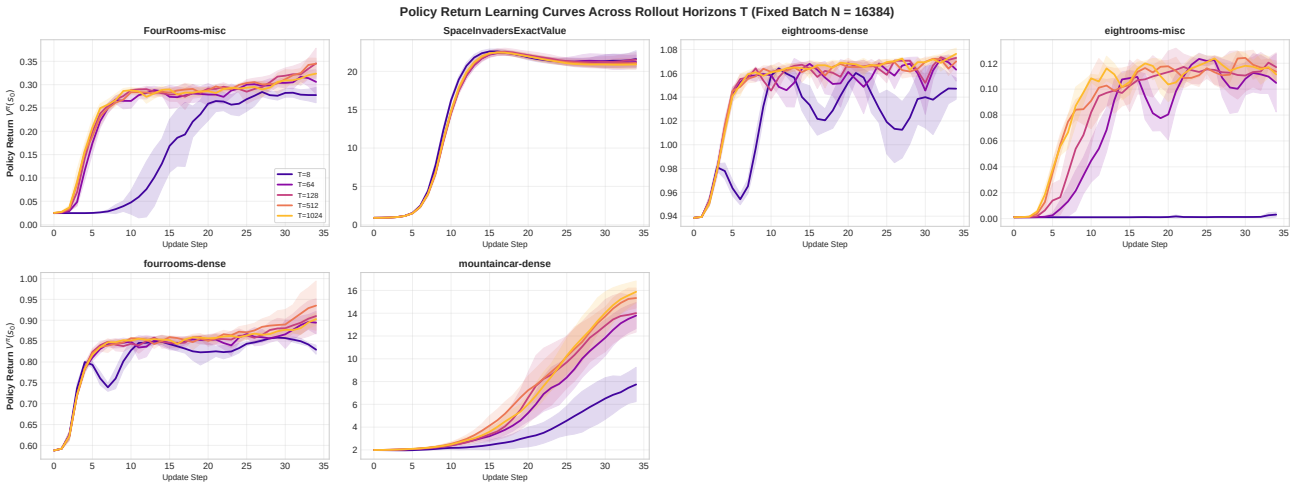


Figure 3. **Performance of PPO using E-Based Critic by Horizon** Performance of PPO with value learning due to the (sampled) Bellman Smoothed Value Loss for different horizon values T. Each horizon still has the same number of samples and gradient steps per update.

on the sampled data $\hat{\boldsymbol{g}} = \frac{1}{TB}\sum_{b=1}^{B}\sum_{t=1}^{T}\nabla_\theta l_e(s_{b,t}, s_{b,t+1}, G_{b,t}^T, G_{b,t+1}^T)$. Both $\hat{\boldsymbol{g}}$ and $\boldsymbol{g}^*$ are length k vectors in paramter space. The MSE $\|\hat{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2$ is the sum of the squared truncation bias and variance.

Figure **??** shows performance during learning for each considered rollout length T, demonstrating that performance generally improved with T.

Figure **??** shows estimated Bias^2 and Variance, finding that generally the bias decreases with T, while variance increases.

Figure **??** shows the mean squared error of the gradient $\|\hat{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2$ during learning.

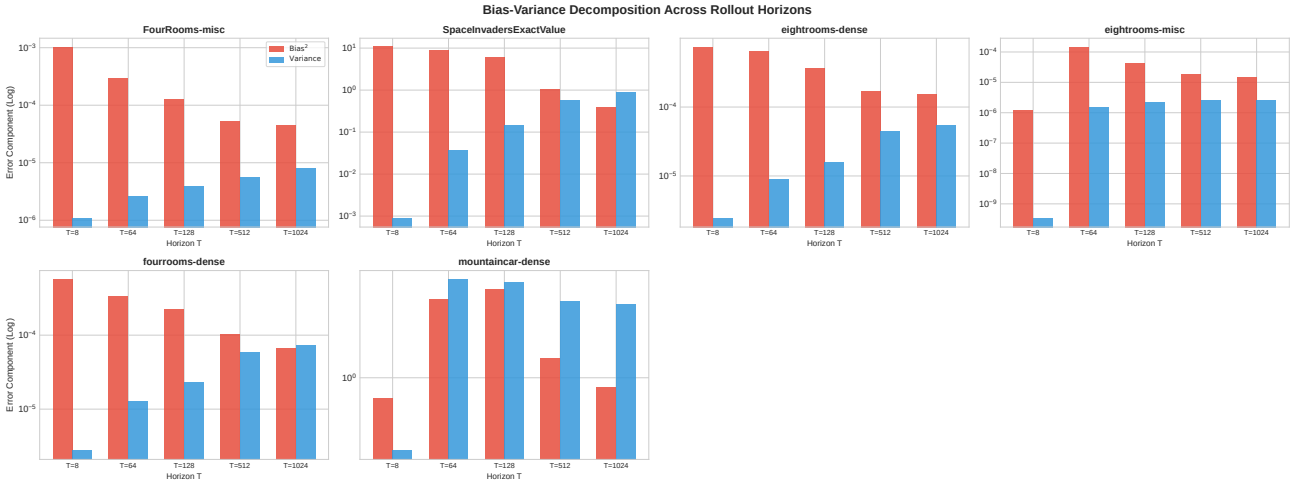


Figure 4. **Bias-Variance Trade-off in Bellman Error Gradient Estimation** ($N = B \times T = 16{,}384$)**.** Decomposition of the total gradient mean squared error $\|\hat{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2 = \text{Bias}^2 + \text{Variance}$ across rollout horizons $T \in \{8, 64, 128, 512, 1024\}$. As the rollout horizon T increases, truncation bias strictly decreases as the T-step return approximation $\delta_i^T \to e_i$ becomes exact. Concurrently, sample variance increases due to the reduced number of parallel trajectory rollouts $B = N/T$ under a fixed computational budget. Note that there is bias in the estimate of Bias^2 itself, preventing it from going to zero.

C. Deep RL Experiments

Bias-Variance Decomposition of Gradient Estimates

Let $\boldsymbol{g}^* = \mathbb{E}_{\mu^\pi}[\nabla_\theta \mathcal{E}]$ denote the exact expected Bellman error gradient in parameter space $\mathbb{R}^k$, and let $\hat{\boldsymbol{g}} \in \mathbb{R}^k$ denote the empirical gradient computed from a batch of B rollout trajectories of horizon T.

1. Population Bias-Variance Decomposition

Defining the population expectation of the sampled gradient as $\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} \triangleq \mathbb{E}[\hat{\boldsymbol{g}}]$, the total Mean Squared Error (MSE) decomposes into the squared truncation bias and the trace of the gradient covariance matrix:

$$\begin{aligned}
\mathrm{MSE}(\hat{\boldsymbol{g}}) &\triangleq \mathbb{E}\left[\|\hat{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2\right] \\
&= \mathbb{E}\left[\| \left(\hat{\boldsymbol{g}} - \boldsymbol{\mu}_{\hat{\boldsymbol{g}}}\right) + \left(\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} - \boldsymbol{g}^*\right) \|_2^2\right] \\
&= \|\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} - \boldsymbol{g}^*\|_2^2 + \mathbb{E}\left[\|\hat{\boldsymbol{g}} - \boldsymbol{\mu}_{\hat{\boldsymbol{g}}}\|_2^2\right] + 2 \underbrace{\mathbb{E}\left[\hat{\boldsymbol{g}} - \boldsymbol{\mu}_{\hat{\boldsymbol{g}}}\right]^\top}_{=\mathbf{0}} \left(\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} - \boldsymbol{g}^*\right) \\
&= \underbrace{\|\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} - \boldsymbol{g}^*\|_2^2}_{\text{Truncation Bias}^2} + \underbrace{\mathrm{Tr}\left(\mathrm{Cov}(\hat{\boldsymbol{g}})\right)}_{\text{Sample Variance}}.
\end{aligned} \tag{22}$$

2. Empirical Sample-Level Decomposition

In empirical experiments, we evaluate M independent stochastic runs (seeds) $\{\hat{\boldsymbol{g}}^{(m)}\}_{m=1}^M$ under identical policy parameters, and compute the sample mean gradient:

$$\bar{\boldsymbol{g}} = \frac{1}{M} \sum_{m=1}^{M} \hat{\boldsymbol{g}}^{(m)}. \tag{23}$$

By Huygens' sum-of-squares identity, the total empirical MSE partitions exactly into two non-negative terms:

$$\underbrace{\frac{1}{M} \sum_{m=1}^{M} \|\hat{\boldsymbol{g}}^{(m)} - \boldsymbol{g}^*\|_2^2}_{\widehat{\mathrm{MSE}}} = \underbrace{\|\bar{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2}_{\widehat{\mathrm{Bias}}^2_{\mathrm{sample}}} + \underbrace{\frac{1}{M} \sum_{m=1}^{M} \|\hat{\boldsymbol{g}}^{(m)} - \bar{\boldsymbol{g}}\|_2^2}_{\widehat{\mathrm{Var}}_{\mathrm{sample}}}. \tag{24}$$

This algebraic identity guarantees that $\widehat{\mathrm{MSE}} \equiv \widehat{\mathrm{Bias}}^2_{\mathrm{sample}} + \widehat{\mathrm{Var}}_{\mathrm{sample}}$ strictly holds across all seeds and horizons without leaving an unassigned residual or generating negative quantities on logarithmic plots.

3. Finite-Sample Expectation of the Estimator

Taking the expectation of the empirical plug-in bias estimator $\widehat{\mathrm{Bias}}^2_{\mathrm{sample}}$ reveals that it is a biased estimator of the true population squared bias $\|\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} - \boldsymbol{g}^*\|_2^2$:

$$\begin{aligned}
\mathbb{E}\left[\widehat{\mathrm{Bias}}^2_{\mathrm{sample}}\right] &= \mathbb{E}\left[\|\bar{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2\right] \\
&= \mathbb{E}\left[\| \left(\bar{\boldsymbol{g}} - \boldsymbol{\mu}_{\hat{\boldsymbol{g}}}\right) + \left(\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} - \boldsymbol{g}^*\right) \|_2^2\right] \\
&= \|\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} - \boldsymbol{g}^*\|_2^2 + \mathbb{E}\left[\|\bar{\boldsymbol{g}} - \boldsymbol{\mu}_{\hat{\boldsymbol{g}}}\|_2^2\right] \\
&= \|\mathrm{Bias}(\hat{\boldsymbol{g}})\|_2^2 + \frac{1}{M} \mathrm{Tr}\left(\mathrm{Cov}(\hat{\boldsymbol{g}})\right).
\end{aligned} \tag{25}$$

Consequently, due to the convexity of the L_2 norm (Jensen's inequality), $\widehat{\mathrm{Bias}}^2_{\mathrm{sample}}$ carries an unavoidable additive variance penalty of $\frac{1}{M}\mathrm{Var}(\hat{\boldsymbol{g}})$. Even in the asymptotic limit where truncation bias vanishes ($\boldsymbol{\mu}_{\hat{\boldsymbol{g}}} \to \boldsymbol{g}^*$ as $T \to \infty$), the empirical sample bias hits a noise floor of $\frac{1}{M}\mathrm{Var}(\hat{\boldsymbol{g}}) > 0$.

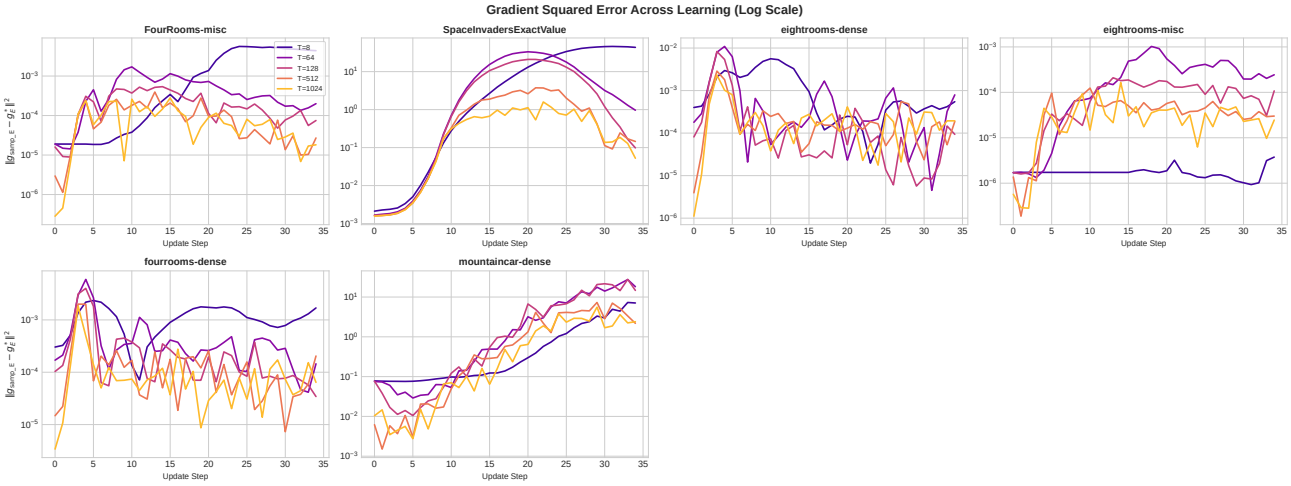
An unbiased estimator of population squared bias is given by:

$$\widehat{\mathrm{Bias}}^2_{\mathrm{unbiased}} = \|\bar{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2 - \frac{1}{M-1} \widehat{\mathrm{Var}}_{\mathrm{sample}}, \tag{26}$$

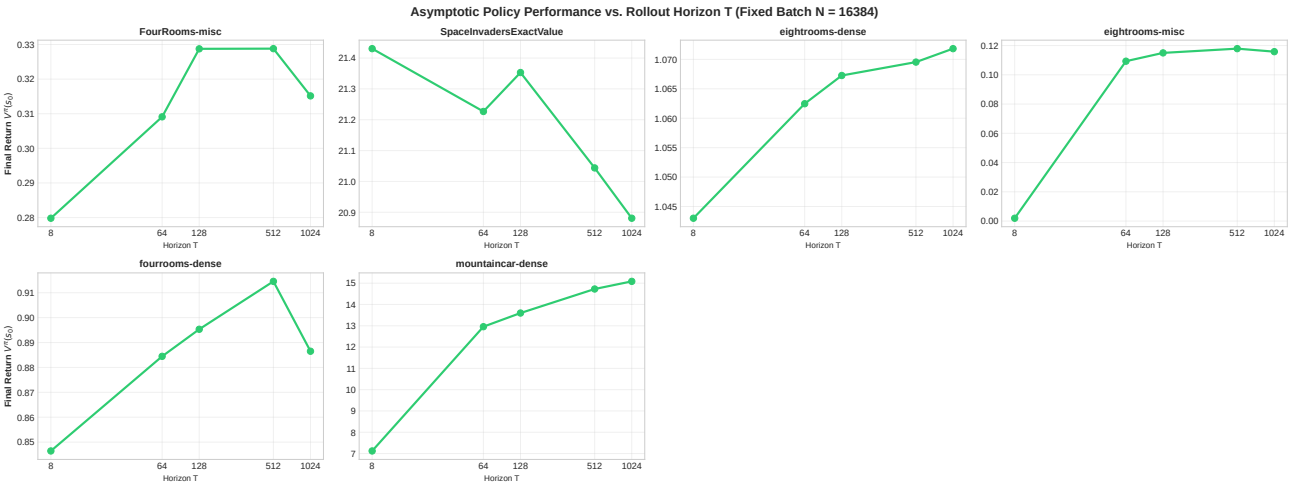
which subtracts the finite-sample variance penalty at the expense of potentially admitting negative values under finite sample noise when the true bias is negligible.

You can have as much text here as you want. The main body must be at most 8 pages long. For the final version, one more page can be added. If you want, you can use an appendix like this one.

The `\onecolumn` command above can be kept in place if you prefer a one-column appendix, or can be removed if you prefer a two-column appendix. Apart from this possible change, the style (font size, spacing, margins, page numbering, etc.) should be kept the same as the main body.



(a) Gradient Squared Error $\|\hat{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2$ Across Learning Updates (Log Scale)



(b) Asymptotic Policy Performance $V^{\pi}(s_0)$ as a Function of Rollout Horizon T

Figure 5. **Gradient Error and Policy Performance by Rollout Length. (a)** The empirical gradient MSE $\|\hat{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2$ over policy updates across 7 environments. Shorter horizons suffer from persistent truncation bias floors, while longer horizons ($T \geq 512$) drive gradient error orders of magnitude lower. **(b)** Final policy return $V^{\pi}(s_0)$ vs. horizon T, demonstrating that reducing truncation bias usually improves downstream policy optimization.

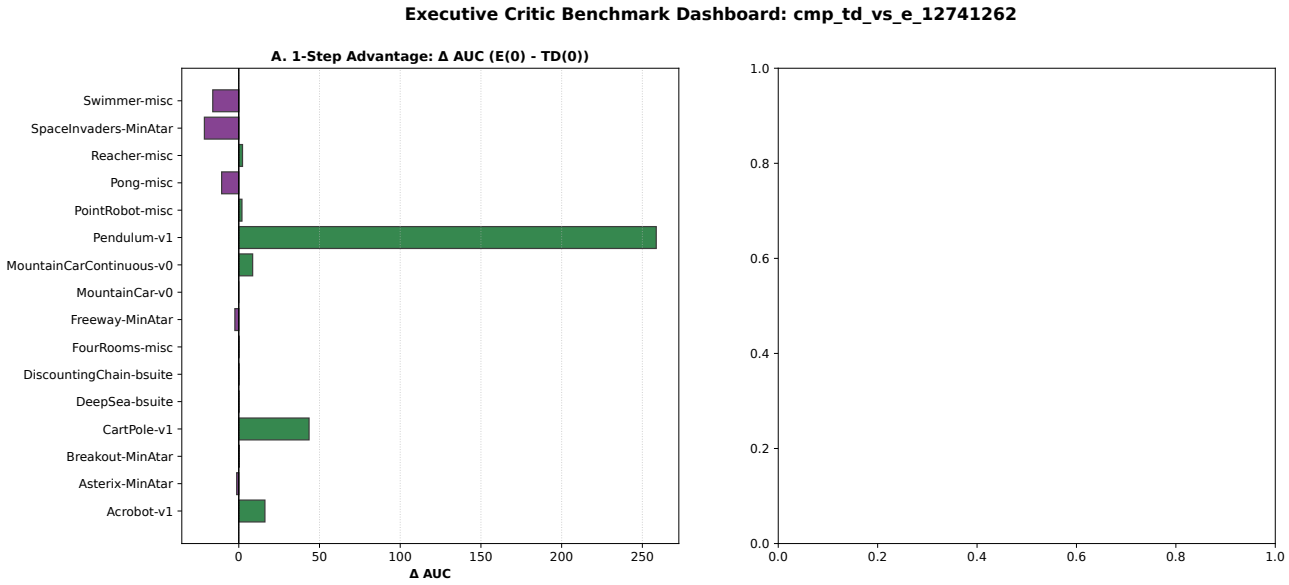


Figure 6. **Gymnax TD(0) vs E(0).** Decomposition of the total gradient mean squared error $\|\hat{\boldsymbol{g}} - \boldsymbol{g}^*\|_2^2 = \text{Bias}^2 + \text{Variance}$ across rollout horizons $T \in \{8, 64, 128, 512, 1024\}$. As the rollout horizon T increases, truncation bias strictly decreases as the T-step return approximation $\delta_i^T \to e_i$ becomes exact. Concurrently, sample variance increases due to the reduced number of parallel trajectory rollouts $B = N/T$ under a fixed computational budget. Note that there is bias in the estimate of Bias^2 itself, preventing it from going to zero.